Improving I/O Performances of HPC Application Gysela

Alexandre Lou-Poueyou, Francieli Boito, Meline Trochon, Luan Teylo

TECHNICAL REPORT
N° XXXX
MONTH YEAR
Project-Teams TADaaM

ISSN 0249-0803   ISRN INRIA/RT--XXXX--FR+ENG

Improving I/O Performances of HPC Application Gysela

Alexandre Lou-Poueyou[*], Francieli Boito[†], Meline Trochon[‡], Luan Teylo [§]

Project-Teams TADaaM

Technical Report n° XXXX — MONTH YEAR — 7 pages

Abstract: In modern High-Performance Computing (HPC) systems, Input/Output (I/O) performance is a major bottleneck for large-scale scientific simulations. As computational power and data volumes scale rapidly, optimizing I/O workflows becomes critical to efficiently leverage supercomputing resources. Scientific applications heavily rely on Parallel File Systems (PFS) to access vast datasets concurrently. However, achieved performance is highly sensitive to PFS configurations, high-level I/O strategies, and application-specific access patterns. Understanding and tuning the interactions between these layers is crucial for maximizing global simulation throughput.
This work focuses on optimizing the checkpointing performance of Gysela, a 5D gyrokinetic simulation code developed by the CEA for nuclear fusion research. Gysela generates massive datasets and relies on periodic checkpointing for fault tolerance and job restartability. Using gys_io, a lightweight mini-app replicating Gysela's parallel I/O patterns, we systematically evaluate and compare different parallel I/O strategies—namely Single Shared File (SSF), Subfiling, and File Per Process (FPP)—alongside key PFS configuration parameters. Our experiments, conducted on diverses clusters, analyze the performance trade-offs of these strategies and parameters under various scaling conditions. The results identify optimal parallel configurations, clarify the impact of PFS striping on each I/O strategy, and provide actionable optimization recommendations to significantly reduce checkpointing overhead in future Gysela production runs.

Key-words: HPC, Parallel I/O, Performances Tuning, Parallel File Systems

[*] Computer Science Master's Degree Student at University of Bordeaux - Intern at TADaaM team
[†] Associate Professor at University of Bordeaux & permanent researcher at TADaaM team
[‡] PhD stutent at TADaaM team
[§] Permanent researcher at TADaaM team

RESEARCH CENTRE
Centre Inria de l'Université de Bordeaux

200 avenue de la Vieille Tour
33405 Talence Cedex

Contents

1 Introduction

In modern High-Performance Computing (HPC) systems, Input/Output (I/O) performance has become nearly critical as computational power itself. As supercomputer advance toward exascale capabilities, computational throughput has increased exponentially, yet storage bandwidth has lagged behind. This growing imbalance creates a fundamental bottleneck: while modern processors achieve teraflop- scale performance, parallel file systems typically deliver only tens to hundreds of gigabytes per second in aggregate bandwidth. Figure 1 quantifies this imbalance using data from major supercomputers from 2000 to 2025. The I/O bandwidth per unit of compute (measured in GBps per PetaFlop) has declined from ∼300 GBps/PF on ASCI Red (2000) to only ∼6 GBps/PF on Frontier (2023) a 50-fold degradation. This trend accelerated sharply after 2015, demonstrating that I/O has become the dominant bottleneck for modern exascale systems.

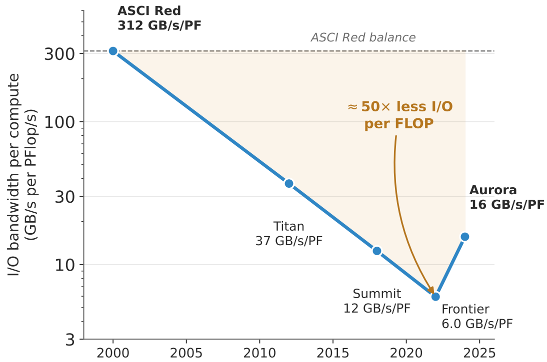

Figure 1: Compute-to-I/O performance gap in top 500 supercomputers (2000–2025).
The ratio of computational throughput to storage bandwidth has grown by an order of indicating that I/O is increasingly the limiting factor.

Large-scale scientific simulations generate massive data volumes requiring efficient I/O management. On parallel file systems, thousands of processes must coordinate concurrent access to shared storage while contending for limited bandwidth. Gysela is a 5D gyrokinetic simulation code developed by the CEA (Commissariat à l'énergie Atomique) for nuclear fusion research. It models plasma turbulence in tokamaks by solving the Vlasov equation in a five-dimensional phase space: three spatial coordinates (minor radius and two toroidal angles) and two velocity-space coordinates (parallel velocity and magnetic moment). Understanding and controlling the turbulence that degrades plasma confinement is crucial for the success of next-generation fusion reactors like ITER. Production Gysela simulations run thousands of processes and generate massive checkpoint datasets. Due to the long execution times (ranging from days to weeks on large allocations) and the constant risk of hardware failures or job time limits on production systems, Gysela relies on periodic checkpointing saving the complete simulation state at regular intervals to enable clean restarts without recomputing from the beginning. The checkpointing writing process is synchronous: All MPI ranks pause computation and collectively write their portion of the 5D state to persistent storage. During this phase, expensive supercomputer resources sit idle, wasting valuable compute hours. In production environments, checkpoints write time suffers from significant variability-sometimes exceeding the mean by a factor of 3 or more-due to PFS contention with other jobs and variable network behavior. This unpredictability difficult to accurately predict job completion times and optimize checkpoint frequency. Optimizing checkpoint I/O requires simultaneous attention to two distinct challenges: tunin parallel file system configuration parameters (e.g., stripe count, stripe size and OST allocation) detailled in section 3 and selecting an appropriate I/O strategy (how application data is organized and written to the PFS). To facilitate rapid, reproducible experimentation, we use `gys_io`, a lightweight mini-application that replicates Gysela's 5D checkpoint pattern without the computational burden. This mini-app enables benchmarking to complete in seconds per configuration rather than hours, allowing systematic parameter sweeps across hundreds of experiments. Importantly, `gys_io` accepts the same configuration files as Gysela, ensuring that benchmark results directly inform production deployments. Optimizing checkpoint I/O requires tuning both parallel file system configuration parameters and selecting an appropriate I/O strategy. This work evaluates three complementary approaches and identifies the most effective configurations for Gysela.

2 Context

This work is conducted at Inria, the French National Institute for Research in Digital Science and Technology. Inria is internationally recognized for its major contributions to high-performance computing, artificial intelligence, and computer networks. With ten research centers across France and over 3,500 researchers, engineers, and administrative staff, Inria leads fundamental and applied research in computer science and mathematics with a focus on societal impact.

This intership is part of the TADaaM team (Topology-Aware system-scale Data management for high-performance computing) at the Inria Bordeaux research center. TADaaM is dedicated to designing a comprehensive service layer for HPC systems, enabling applications to express their resource requirements and allowing the system to dynamically optimize execution at scale. The team's work bridges application needs and system capabilities, ensuring efficient utilization of large-scale computing resources.

This research is part of the NumPex project, a European HPC research initiative dedicated to preparing Europe's scientific computing ecosystem for the exascale era. NumPex brings together leading research institutions, HPC centers, and industry paterns to develop and validate technologies for extreme-scale simulation and data processing. This work contributes to Exadost, a NumPex work package focused on optimizing I/O performance and data management strategies for exascale applications.

The Gysela application studied in this work is developed by the CEA (Commissariat à l'énergie Atomique et aux énergies alternatives), France's atomic and alternative energies commission. Gysela is used by CEA researchers to study plasma turbulence in fusion reactors, making I/O optimization critical for advacing fusion energy research on future exascale systems.

section 1 on page 4.

3 Background

3.1 Parralel File System

- Difference between File System and Parallel File System is striping concept
- Multiple PFS exist : BeegFS, Lustre, ...
- OST stand for Object Storage Target
- OST is a physical and logical unit measure represented by hard disk or group of disk
- Allows data to be partitioned across multiple storage systems

- Striping is a data distribution technique
- Cut datas into multiple chunk called stripe
- Those chunks are written on OSTs
- Main objective of PFS is to improve bandwidth usage

3.2 Write Strategy

We implement three distinct I/O strategies Single Shared File (SSF), Subfiling and File Per Process (FPP) using PDI (Parallel Data Interface), a middleware layer developped by `Maison de la simulation` team that decouple I/O strategy from application code. Each strategy is selected by modifying a YAML configuration file; no code recompilation is required. This flexibility allows rapid evaluation of strategies and enables production systems to adapt I/O policies without code maintenance overhead.

3.2.1 Single Shared File

- Every processes write on a same file
- Two-Phase I/O
- First Data aggregation and then send to PFS

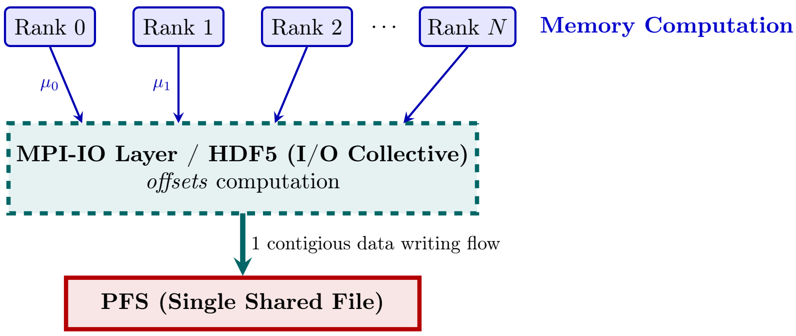

Figure 2: Illustration of the Single Shared File (SSF) strategy.

3.2.2 Subfiling

- Processes's data are aggregated into a smaller number of files called subfiles
- Each subfile is written to the PFS by a single process (IO Concentrator)
- Number of subfiles is a tunable parameter that can be adjusted to optimize performance based on each applications.
- Writing phase of subfiles is done by chunks of data called stripes size 3.1 that are written in parallel on the PFS
- Subfiling strategy aims to reduce contention on the PFS by decreasing the number of concurrence write operations.

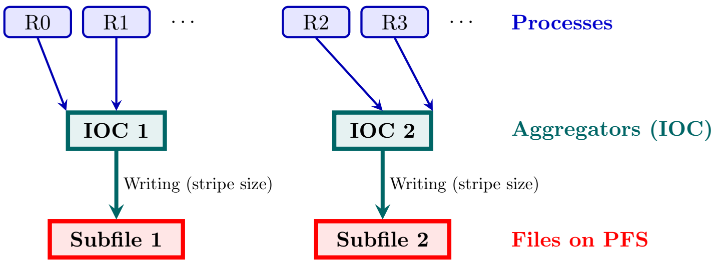

Figure 3: Illustration of the Subfiling strategy.

3.2.3 File Per Process

- Each process writes its own file on the PFS
- This strategy is simple to implement and does not require any data aggregation or coordination between
- However, it can lead to a large number of small files, which may cause performance issues on some PFS due to metadata overhead and file system limitations.

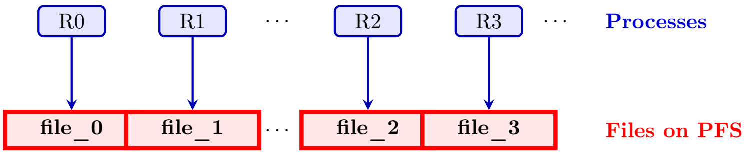

Figure 4: Illustration of the File Per Process (FPP) strategy.

3.3 Software Stacks & Tools

3.3.1 Gysela Mini App IO

- Gysela is

3.3.2 PDI

3.3.3 IOPS

3.3.4 TOTO

4 Experimental Set-up

RESEARCH CENTRE
Centre Inria de l'Université de Bordeaux

200 avenue de la Vieille Tour
33405 Talence Cedex

Publisher
Inria
Domaine de Voluceau - Rocquencourt
BP 105 - 78153 Le Chesnay Cedex
inria.fr

ISSN 0249-0803